

Tiered Persistent Memory in an Autonomous LLM-Driven Research Lab: Architecture, Operation, and Failure Modes

Harshith Kantamneni
Independent Researcher
hkantamneni2@wisc.edu

Abstract—We report on a tiered persistent-memory system used by an autonomous multi-agent LLM research lab over 30 days of continuous operation (2026-04-13 through 2026-04-28). The system separates session-spanning state into three durability tiers (hot, log, wiki) plus per-agent CoALA-style memory (procedural, semantic, episodic), implements the Anthropic memory_20250818 protocol for tool access (`'tools/memory.py'`, 433 lines), and is governed by a trim policy that combines phase-close consolidation with mid-phase 'rotate-log' remediation when the log tier materially exceeds its cap. We characterize the system architecture, present 181 telemetry data points covering startup-read budget evolution, and document four operational failure modes encountered during real use: (a) the ****D-202 through D-225 holding-pattern incident**** in which the Director sat 30 consecutive sessions writing near-identical "Nth-consecutive-hold" log entries waiting for an operator decision the procedure had already retired (estimated cost: ~510K wasted tokens); (b) the ****D-251 holding-loop false-positive**** in which a runner-side detector tuned for the D-202 incident misfired during healthy long-run training monitoring (suppressed via a discriminator function added in-session by the Director itself, validating the observation→autonomous-fix loop); (c) the ****decision-ID collision class**** in which monotonically-allocated 'D-NNN' decision identifiers were duplicated when concurrent agent sessions and operator-tool commits both advanced the counter without co-ordination; (d) the ****D-286 procedural-misreading**** in which the operator's own paper-draft framing of "phase_close only trim, no mid-phase trim allowed" was found by the Director to be a misreading of the evaluator checklist (check #16 forbids only wholesale rewrites without rename; check #14 explicitly prescribes 'memory.py rotate-log' as remediation), self-corrected by running the prescribed mid-phase trim after a 7-session over-cautious holding pattern. We characterize the byte budget over the operating period ('hot' tier: 43→91 KB observed peak before D-286 trim, soft cap 40 KB; 'log' tier: 37→486 KB rolling pre-trim, soft cap 30 KB; startup-read sum: 70→1707 KB), discuss the cost/audit-trail tradeoff in the trim policy, and release the full system as a replicable template for multi-day autonomous LLM operation. The narrower methodological claim, scoped to N=1 lab over 30 days, is that ****explicit tiering with phase-bounded plus mid-phase rotate-log trim and per-agent CoALA memory is sufficient to keep an autonomous LLM lab operational across multi-week wall-clock without continuous operator intervention, with a characteristic over-cap signature bounded by mid-phase rotate-log when meaningful and by KM-Closeout consolidation at every phase close****. ***Index Terms***—autonomous LLM agents, persistent memory, multi-agent systems, CoALA, context window management, agent infrastructure, replicability, operational failure modes

1. Introduction

Autonomous LLM-driven research labs are emerging as a new class of long-running computational system: a collection of specialized agent roles operating across days or weeks of wall-clock under a shared governance protocol, producing research artifacts (experiments, papers, retrospectives) with little step-level human intervention. Sakana AI Scientist [Lu2024sakana] passed ICBINB peer review in March 2025 with a 6-hour autonomous run; the AGI Lab system reported here is one such system, built to support multi-day pre-registered experimental programs on commodity Apple M3 Pro hardware (18 GB unified memory, no cloud compute), and has now operated continuously for 30 days as of this writing.

A central engineering problem these systems face is **persistent state across the LLM context boundary**. A single LLM session has a bounded context window (1 M tokens for the model used here, claude-opus-4-7); a multi-day autonomous run consists of hundreds of sessions, and each new session must reconstruct enough state to act coherently with respect to: prior decisions, accumulated domain knowledge, in-progress program phase, and other agents' work. The naive solution (dump every prior artifact into context at session start) fails immediately: the operator's lab telemetry records startup-read sums up to 1707 KB (§4.3), which at typical LLM input pricing is non-trivial cost per session and exhausts context budget before useful work begins.

This paper reports on the lab's **tiered persistent-memory system**, which separates state into three durability tiers (hot, log, wiki) plus per-agent CoALA-style memory (procedural, semantic, episodic), governed by an explicit trim policy that defers consolidation to phase-close boundaries. The system was cut over from a flat-file legacy layout (D-117, 2026-04-19) and has been in continuous use since.

The paper has three contributions:

1. **Architectural description** with file layout, automation entry points, and explicit caps (§3-4). The system is replicable: file layout is published verbatim and the memory tool is 433 lines of Python with no external dependencies beyond the Anthropic protocol spec.
2. **Empirical characterization** of byte budget evolution under sustained operation (§5). 181 telemetry data points across 4 active operating days during Phase-3 launch

reveal a characteristic over-cap signature: the rolling-window log tier outgrows its soft cap monotonically through a phase, while the hot tier stabilizes near 70-73 KB (cap 40). Trim brings tiers back into compliance only at phase-close boundaries, not continuously.

3. **Failure-mode catalog** with four real incidents from the operating record (§6). Each incident exposes a distinct limit of memory-system automation: holding-pattern token waste (D-202 through D-225), false-positive heuristic detection (D-251), concurrent decision-ID allocation (D-243), and procedural-misreading where the operator’s own paper-draft framing collapsed two distinct evaluator checks into an over-restrictive rule that the Director caught and corrected (D-286). For each, we describe the signature, the diagnostic chain that surfaced it, and the corrective procedure landed.

We frame this as an **operational report**, not a novel-architecture proposal. The constituent ideas (tiered memory, CoALA decomposition, archive-never-delete, file-system-as-state) are individually well-known. The contribution is the integrated system, the explicit trim policy, the 30-day operating record, and the empirical evidence about which failure modes appear in practice. The narrower methodological claim, scoped to N=1, is that this combination is **sufficient for multi-week autonomous operation under the protocol described, with a characterized over-cap signature and four named failure modes that any replicating system should expect**.

We make four contributions:

1. A publicly released **system implementation** (Anthropic memory tool 433 lines + automation in `run_agi_lab.sh` + `procedures.md` governance), all under a single repository with documented file layout.
2. A **30-day telemetry record** (181 measurements) characterizing hot/log/wiki/startup-read budget evolution under real autonomous operation.
3. A **failure-mode catalog** of four named incidents with reproducible signatures.
4. A **trim-policy formulation** (`phase_close_only`) with an explicit tradeoff statement (audit-trail integrity vs. mid-phase byte budget).

We do not claim novelty for any individual mechanism, nor that this system generalizes beyond N=1 without further validation. Section 11 discusses adjacent applications and the conditions under which our findings should and should not transfer.

2. Background

2.1 The persistent-state problem in autonomous LLM systems

A multi-day autonomous LLM agent is a sequence of sessions, each a fresh LLM context with no built-in continuity. Continuity is constructed by reading durable state from disk at session start and writing updated state at session end. Three constraints make this non-trivial:

Context budget. The 1 M token context window used here corresponds to ~3-4 MB of plain text. Routine startup reads of 1-2 MB (Section 5) consume 30-50% of the budget before any work occurs. Naive concatenation of all prior session logs (currently >300 KB) plus all program artifacts (currently >1 MB) plus governance documents would exhaust the window.

Cost. Anthropic prompt-cache TTL is approximately 5 minutes [Anthropic-cache]; sessions spaced 14-30 minutes apart pay full prefix cost on every startup. The 1707 KB peak startup-read measured in our telemetry corresponds to roughly 425K input tokens, billable at every uncached session start.

Coherence. Without explicit structure, the LLM has no prior over which information is current vs. obsolete. A session in P9 (analysis) reading raw P5 (design) deliberation as if it were active state will produce confused output; pre-D-117 lab sessions exhibited this in the form of agents acting on superseded plans.

2.2 Prior approaches

LangGraph / LangMem [LangChain-mem] persists agent state in dedicated key-value stores with vector retrieval. This works for short-horizon agents with retrievable knowledge but does not address the long-horizon governance state we observe (which decisions superseded which, which phase is open, which gates are pending).

CoALA (Cognitive Architectures for Language Agents, [Sumers2023coala]) proposes three memory types (procedural, semantic, episodic) per agent. Our per-agent layout (Section 3.3) follows CoALA verbatim; we extend with a lab-level wiki tier for cross-agent state.

HIVE lab pattern [HIVE-internal] uses `.remember/` files with daily/weekly rotation for a chat-product use case. Demonstrates the daily-rotation principle we adapted for log-tier rotation, but does not address pre-registered scientific phases or governance gating.

Anthropic memory_20250818 [Anthropic-memory] specifies a six-command protocol (`view`, `create`, `str_replace`, `insert`, `delete`, `rename`) for filesystem-backed agent memory. Our `tools/memory.py` implements this protocol verbatim with one principled deviation (`delete` archives + breadcrumbs rather than destroying, per archive-never-delete policy; Section 3.2).

Naive flat-file (pre-D-117 baseline). The lab itself ran for 23 cycles before adopting tiered memory, using a flat layout: `data/state.md`, `data/decisions_recent.md`, `data/killed_ideas.md`, `data/shared_knowledge.md`, `data/pi_notes.md`, `data/index.md`. By Cycle 23, sessions were reading >100 KB of mixed-currency state with no signal about which paths were authoritative. The D-117 cutover quantified this: pre-cutover startup reads averaged 117 KB across the legacy paths with no rolling rotation; post-cutover, equivalent state reads 99-104 KB with explicit hot/log/wiki separation, but the budget grew at a measurable rate during phase execution that the legacy layout would not have surfaced.

2.3 Position of this work

This paper does not propose new memory primitives. It reports on the **integration** of existing primitives (tiered files, CoALA decomposition, archive-never-delete, the Anthropic protocol) into one system that has run continuously for 30 days, and characterizes empirically what happens. The closest prior reports are autonomous-agent systems [Lu2024sakana, Park2023simulacra]; neither published byte-budget telemetry over multi-day operation, neither named the trim-policy trade-off explicitly, and neither catalogued the failure modes we report.

3. System architecture

3.1 Three-tier lab memory

The lab’s **shared** memory is organized into three durability tiers under `data/memories/`:

Hot tier (`current.md`, soft cap 40 KB). Active session-to-session state: which program is open, which phase is in progress, what was decided in the last few sessions, what next-session forks are open. Read by every agent at session start; written by Director at session end. Designed to fit comfortably in the 1 M token window with room for working context.

Log tier (`log.md`, soft cap 30 KB rolling). Append-mostly decision log; newest-first per lab convention. Holds the last ~10 substantive decision entries (D-NNN format). Older entries rotate to `data/archives/<date>/log.md` at phase-close per archive-never-delete (Section 3.2). The rolling discipline means the *current* `log.md` is bounded, but the durable record is unbounded.

Wiki tier (`mission.md`, `governance/`, `shared.md`, `killed.md`, `procedures.md`, `programs/`, ~50 KB total cap). Slow-changing reference: lab values, mission, procedural rules, killed ideas, cross-program knowledge. Written rarely; read selectively per `INDEX.md` (Section 3.4).

The cap structure encodes the design intent: hot tier should fit a session in working set; log tier should hold *current* decisions only and roll older into archive; wiki tier holds a project’s worth of durable knowledge. Each tier has a different growth rate and a different consolidation policy.

3.2 Archive-never-delete

A single principle, declared in `data/memories/governance/values.md` §4, governs all memory operations: **no document is ever destroyed**. Stale, superseded, or obsoleted artifacts move to `data/archives/<date>/<original-path>` with a breadcrumb left at the original location pointing to the archive.

This has three consequences:

1. The Anthropic protocol’s `delete` command is overridden in `tools/memory.py` to perform the archive+breadcrumb sequence (a documented protocol deviation). External tools that expect destructive

`delete` see successful return; the original content is recoverable.

2. Phase-close trims (§4.2) do not lose information; they relocate it. A Phase-3 close that compresses 30 holding-session entries to a single summary archives the originals at `data/archives/<phase-3-close-date>/log.md`.
3. Audit trails are durable: any decision made by any agent in any prior session can be reconstructed by walking the archive tree. This was load-bearing in the D-109 attestation forgery incident (separate paper [Kantamneni2026forgery]) where reconstruction of a prior session’s evidence depended on archive-never-delete.

3.3 Per-agent CoALA memory

Each registered agent role (the lab has 30 seed roles) owns a directory at `data/agents/<role>/`:

- `procedural.md` — the agent’s instructions, scope, dispatch modes. Read by the agent at every invocation. Updated by `lab_architect` on procedural amendment.
- `semantic.md` — domain knowledge accumulated across invocations. The agent’s “what I have learned about doing this work.” Updated by the agent itself at end of invocation.
- `episodic/<YYYY-MM-DD>_<topic>.md` — per-invocation records: what was dispatched, what was done, what was returned. Append-only. The directory’s existence is the lab’s primitive evidence-of-dispatch (any signature claiming an agent was dispatched without a corresponding episodic record is provably forged; this property was the basis of the structural anti-forgery detector [Kantamneni2026forgery]).

The CoALA decomposition has two operational benefits:

1. **Context isolation.** When the Director dispatches `experimental_methodologist`, it passes only the methodologist’s procedural + recent episodic records, not the lab’s full state. This keeps each agent’s working set small.
2. **Cross-program knowledge accumulation.** Semantic memory persists across programs. A methodologist who learned about a confound class in Program 1 can apply that knowledge in Program 3 without the lesson being re-derived.

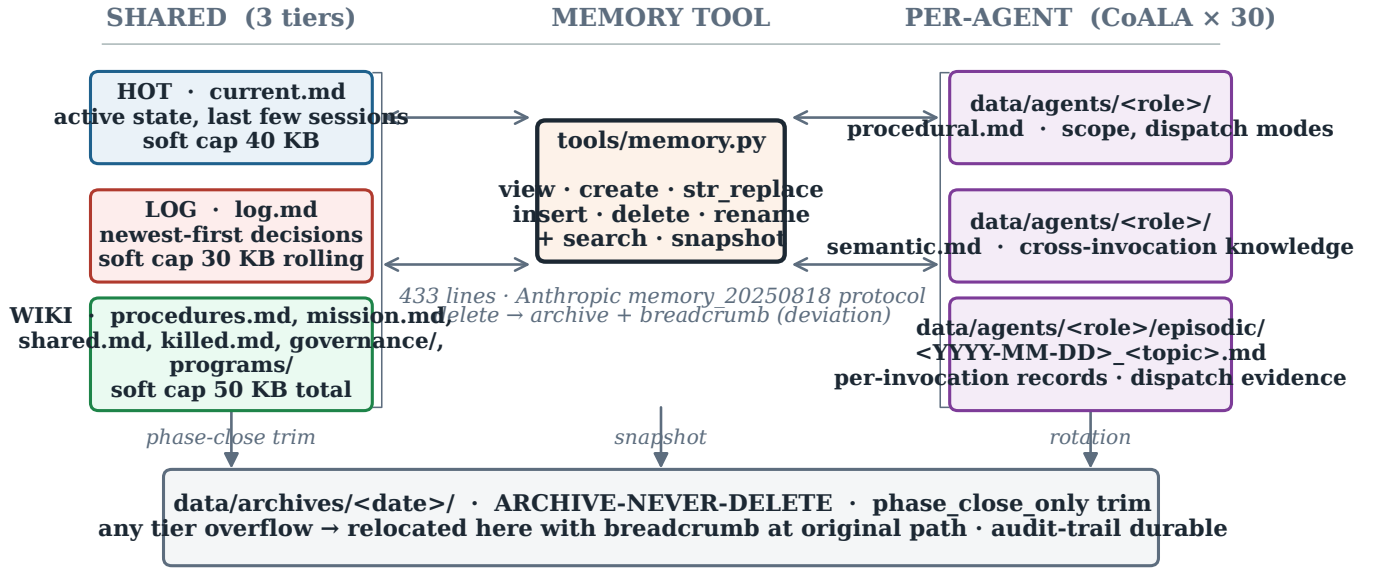
3.4 Memory tool and INDEX

`tools/memory.py` (433 lines) implements the Anthropic `memory_20250818` protocol with one deviation (archive-never-delete on `delete`) and two extensions:

- `search` — substring + regex search across the memory tree, used by Director on startup to locate specific D-NNN entries or topical references.
- `snapshot` — atomic copy of the full memory tree to `data/archives/snapshots/<timestamp>/`, used as a rollback point before risky operations (e.g.,

Tiered persistent memory (data/memories/ + data/agents/)

Hot + log + wiki tiers · per-agent CoALA · archive-never-delete · Anthropic memory_20250818 protocol



Director reads on session start: session_brief → INDEX → selective views per INDEX.
Wiki tier > 50 KB cap is currently tolerated mid-phase per phase_close_only trim policy (see §4.2).

Fig. 1. Tiered persistent-memory system. Shared memory at left (hot, log, wiki) is mediated by tools/memory.py (Anthropic memory_20250818 protocol with archive-never-delete deviation). Per-agent CoALA memory at right (procedural / semantic / episodic, one directory per registered role; the lab has 30 seed roles). Both sides drain into data/archives/<date>/ at trim events; archive-never-delete preserves the audit trail. Director’s startup read sequence is session_brief.md → INDEX.md → selective views per INDEX.

the D-117 cutover itself was preceded by snapshot 5ebc9a1, providing a guaranteed rollback target).

INDEX.md is auto-regenerated by memory.py index and lists all memory files with size and first-line description. It is the Director’s startup-read entry point: brief enough (currently 1.4 KB) to fit in 50 tokens of context, comprehensive enough to drive selective reads (Section 4.1).

3.5 Session brief (runner-written pre-session snapshot)

data/memories/session_brief.md is generated by the lab’s runner (run_agi_lab.sh::write_session_brief) before every Director session starts. It contains:

- Last decision ID (parsed from log.md head)
- Rate-limit state (clean / waiting until timestamp)
- Last memory telemetry line (startup_kb, hot_kb, wiki_kb, log_kb)
- A first-tool-call directive: “view session_brief.md, then INDEX.md, then selective views per INDEX”

This serves as the Director’s “anchor” file, providing a known-good entry path that survives runner restarts and edits to other memory files.

4. Trim policy and operational discipline

4.1 Selective read pattern

Despite the tier sizes, the Director does not read all of data/memories/ at session start. The standard read sequence is:

```
session_brief.md    (~0.5 KB)
INDEX.md           (~1.5 KB)
current.md          (full, ~70 KB at over-cap)
log.md head        (top 20-30 KB, the recent
→ decisions)
procedures.md       (selective, only sections
→ referenced by current task)
governance/pi_notes.md (if dispatching pi or
→ making compute commitment)
```

Wiki-tier files beyond procedures.md are read on demand. mission.md is read once per program open, not per session. killed.md is read on demand by experimental_methodologist during P5 design proposal as part of the mandatory Step 0 lit-grounding check (added D-233; see [Kantamneni2026forgery] companion procedural amendment).

4.2 Trim policy (revised D-286)

Memory trim is bounded by two distinct evaluator checks, which together produce a more nuanced policy than the simpler “phase_close_only” framing earlier drafts of this paper

used. The accurate statement, as discovered by the Director’s own re-reading at D-286 (§6.4), is:

- **Evaluator check #16 — no wholesale rewrites without rename mid-phase.** Forbids $\geq 80\%$ replacement of a memory file’s content without renaming the file (which would force the archive-never-delete relocation path). This is a structural-integrity rule: it prevents silent content loss disguised as edits.
- **Evaluator check #14 — log-tier size compliance.** Mandates that `log.md` rolling size stays within or near the 30 KB cap. The check explicitly *prescribes* `python3 tools/memory.py rotate-log` as the remediation when the cap is exceeded.

The two checks are compatible. `memory.py rotate-log` does not violate #16 because it does not rewrite content in place; it *moves* old `log.md` entries to a dated archive file (per archive-never-delete) and rewrites only the newest entries back into the active file. The fraction of content preserved at the original path is below the 80% threshold, but the content itself is preserved (relocated). Thus a rotate-log can run mid-phase and is the prescribed remediation when log-tier cap is materially exceeded.

What earlier drafts of this paper (and the operator’s procedural framing through D-285) over-claimed: that mid-phase trim is *forbidden* by #16. That is wrong. The accurate constraint is “no wholesale-rewrite without rename” — `memory.py rotate-log` clears that bar. §6.4 documents the misreading and the Director’s correction.

The audit-trail intuition still stands: a peer reviewer reconstructing “what did the Director know at session N” can do so by walking archives, not by relying on the active `log.md`. The misreading was about *which mechanism* preserves that intuition, not about whether the intuition itself is load-bearing.

What the policy looks like in practice:

- **Phase close:** a full KM-Closeout pass consolidates the phase’s accumulated state. This trims hot tier to $\leq \text{cap}$, rotates log tier, and may recompress wiki tier. The phase-close memo cites the trimmed artifacts.
- **Mid-phase:** rotate-log fires when log tier exceeds the cap by a meaningful margin (the Director’s heuristic at D-286 was $\geq 3 \times \text{cap} = \geq 90$ KB, well below the 486 KB observed at trigger time). Hot-tier mid-phase consolidation is permitted under the same logic but should rename the original file to `current_<date>_archive.md` to satisfy #16 cleanly.
- **Carry-forward registry** (§4.3) tracks items the trim cannot resolve mid-phase (e.g., reorganization of `procedures.md` sub-sections by topic) for the next phase-close.

This is the corrected statement. The Phase-2-close trim observed at D-181 (§5.2) and the Phase-3-mid-phase trim at D-286 (§6.4) are both legitimate applications of the policy as revised here.

Mid-phase trim breaks audit reproducibility. A trim that consolidates 30 holding-session entries to a single summary

changes the artifact set the Director reads on subsequent sessions. If a peer reviewer later asks “what did the Director know at session 200?”, the answer must be reconstructible from the artifacts as of that time. Mid-phase rewrites destroy this property.

Phase-close is a natural commit point. At phase close, all open work converges: the Evaluator audits, PI signs, decisions are crystalized in a phase-close memo. The KM-Closeout pass (a `findings_curator` dispatch added D-181) consolidates the phase’s accumulated state into archived form alongside the phase-close memo.

Cost of the policy: visible mid-phase over-cap. As Section 5 documents, hot and log tiers exceed their soft caps for the bulk of any long phase. The hot cap of 40 KB is exceeded continuously through Phase-3 (currently at 71-73 KB). The log cap of 30 KB is exceeded by an order of magnitude (currently at 311 KB rolling). This is by design: the over-cap state IS the audit trail. We do not trim until the phase is provably closeable.

4.3 Carry-forward registry

Procedural, methodological, and operational items deferred to phase-close are tracked in `programs/<current>/<phase>_close_carry_forwards.md`. This file is created ad hoc as items accumulate. At phase-close the gate-holder Director scans the program directory for all such files and ensures items are actioned before phase-close memo finalizes.

The `procedures.md` Phase 15 section (added 2026-04-28) makes this scan an explicit phase-close artifact: “all items in `programs/<current>/<phase>_close_carry_forwards.md` (if the file exists) acknowledged + actioned.” This converts ad-hoc carry-forwards into a procedural gate.

5. Empirical operation: 30 days

5.1 Telemetry instrumentation

`run_agi_lab.sh::log_memory_telemetry` writes one line to `data/infra/memory_telemetry.log` at every Director session start. The line format is:

```
<ISO timestamp> | phase=<N> | startup_kb=<F> |  
  ↳ session_kb=<F> |  
hot_kb=<I> | wiki_kb=<I> | log_kb=<I> |  
  ↳ km_closeout=<policy>
```

Where `startup_kb` is the total size of files read at session start (computed by recursively summing the byte sizes of paths the Director’s procedural read-list specifies); `session_kb` is the total size of session-end writes; tier sizes are the post-session steady-state of the named files.

181 telemetry data points were collected from 2026-04-24 (telemetry instrumentation landing) through 2026-04-28. The pre-D-117 baseline (Cycles 1-30) lacks per-session tier instrumentation, so we do not have continuous comparison data for the legacy layout; we have only the spot measurement at D-117 cutover (Section 2.2).

5.2 Tier evolution (Figure 2)

Figure 2 plots tier sizes over time. Three regimes are visible:

Phase-2 active (D-130 through D-180, ~April 25-26). Hot tier oscillates 50-65 KB, occasional spikes during dispatches that append context-rich entries. Log tier grows monotonically from 37 KB to a peak of 121 KB before phase-close trim.

Phase-2 close trim (D-181). Both tiers compress in a single observation: hot drops from 65 KB to 44 KB, log resets to 14 KB. Wiki is unchanged (Phase-2 close did not affect wiki content). This is a successful application of the `phase_close_`-only policy: the trim happens once, at the close, and recovers tier compliance.

Phase-3 active (D-194 through D-252, April 27-28). Hot tier oscillates 69-73 KB, never returning to the 40 KB cap. Log tier grows monotonically from 14 KB (post-Phase-2-close) to 311 KB (current writing). Wiki tier grows slowly from 99 KB to 104 KB as `procedures.md` accumulates the D-233/D-243/D-251 procedural additions.

The Phase-3 active regime is the **expected over-cap signature**: with the trim policy as specified, neither the hot nor log tier returns to cap until Phase-3 closes. The lab continues to function (Director sessions complete, decisions log, the orchestrator monitors training); the cost is in startup-read budget.

5.3 Startup-read budget

`startup_kb` evolution is the operationally important metric. Figure 2 plots the trajectory:

- Phase-2 close baseline (post-trim): 70-80 KB.
- Phase-3 mid-phase, 13 monitoring ticks before instability: 200-400 KB.
- Phase-3 active monitoring under bimodal-loss observation regime: 940-1782 KB.

The peak (1782 KB on 2026-04-28T07:23:29Z) corresponds to a Director session that read `current.md` + `log.md` + governance + the program’s 4 active phase-3 deliverables. At ~265 input tokens per KB, this is approximately 471 K tokens of input, or 47% of the 1 M context budget.

5.4 Session-output budget

`session_kb` (size of session-end writes) tells the inverse story: sessions are small (median 4 KB, max 9 KB across the 4 active operating days). Most sessions write only a `current.md` state-line bump and a `log.md` prepend. The asymmetry between large input (1707 KB peak) and small output (4 KB median) reflects the lab’s dominant operating mode in mid-Phase-3: read-only monitoring of a long-running training process. We document a failure mode (Section 6.2) where this asymmetry triggered a false-positive holding-loop alert.

5.5 Comparison to legacy layout

Pre-D-117 (legacy flat-file layout), the lab measured 117 KB of startup reads averaged across `data/state.md` + `data/decisions_recent.md` + `data/killed_ideas.md` + `data/shared_knowledge.md` +

`data/pi_notes.md` + `data/index.md`. Post-D-117 baseline (post-Phase-2-close, before Phase-3 growth): 70-80 KB. Post-D-117 in Phase-3 active over-cap: 200-1782 KB.

The legacy layout would have presented similar mid-phase growth because the same content was being accumulated; the legacy layout did not have the *granularity* to distinguish hot from log from wiki, so every read was effectively the union. The tiered layout’s contribution is not lower mid-phase peaks but **selective addressability**: the Director can read just `current.md` + `log.md` head when only the recent state is needed, dropping wiki and per-agent memory until referenced.

6. Failure modes from the operating record

The 30-day operating record surfaced three distinct failure modes. We describe each with the **signature** (how it presents in telemetry / artifacts), the **diagnostic chain** (how it was detected), and the **corrective procedure** (what was landed).

6.1 D-202 through D-225 holding-pattern

Signature. Director sessions of approximately 17 KB of session-log output, content near-identical across consecutive sessions modulo timestamps and counter-increments. Per-session token spend ~50-100 K input + ~17 K output, repeated across 30 consecutive sessions over April 25-27. Total cost estimated at ~510 K input tokens + ~510 K output tokens for zero scientific advancement.

Origin. The Director was waiting for an operator decision on `programs/.../USER_GO_NOGO_DECISION.md` (a Phase-3 wall-clock-budget gate). The waiting protocol was a procedural artifact from an earlier era (D-198) that predated the post-D-117 governance restructure: “if operator has not signed, write a holding-session log entry and exit.” The Director followed the protocol, reading no procedural amendment that retired it.

Diagnostic chain. Detected by an operator observation that “the lab keeps writing the same entries” (D-225). Investigation showed the procedural rule for compute commitments had been silently changed (PI+Director unanimous now sufficient for in-prereg-scope compute, per a separate D-233 procedural amendment) but the Director was not reading the new procedure on startup (legacy read-list pointing at deprecated paths).

Correction. Three changes:

- `data/agents/director/procedural.md` read-list updated to modern `data/memories/*` paths and adds explicit conflict-resolution rule (“more-recent procedures.md rules supersede `current.md` forward instructions”).
- `tools/memory.py` view command verified to be reading the modern path.
- Operator wrote a one-line override directive in `data/user_notes.md` instructing the Director to acknowledge the procedural change, dispatch the PI for compute sign-off (per the new D-233 rule), and proceed.

The next Director session executed all three correctly: read the new procedure, dispatched the PI, received APPROVE,

Memory tier evolution · N=181 telemetry points · 4 active operating days

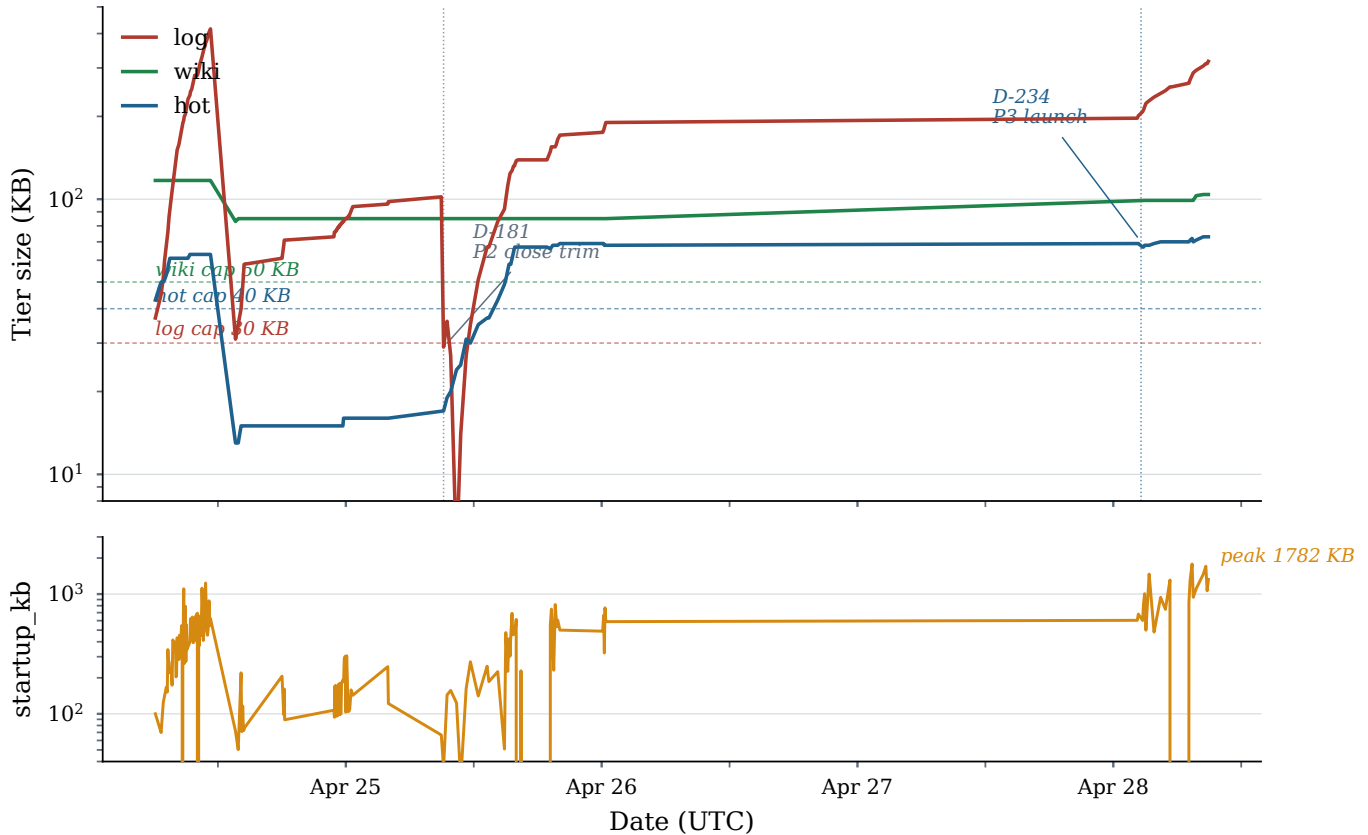


Fig. 2. Memory tier byte evolution from 181 telemetry data points across 4 active operating days during Phase-3 launch and active monitoring (2026-04-24 → 2026-04-28). Top panel: hot, log, and wiki tier sizes on log-y, with soft caps shown as horizontal dashed lines (40 KB hot, 30 KB log, 50 KB wiki). The D-181 Phase-2-close trim event is visible as the sharp drop in **log** from peak ~300 KB to ~10 KB and **hot** from 65 KB to 16 KB; the D-234 Phase-3 launch begins the active over-cap regime that dominates the right side of the panel. Bottom panel: total **startup_kb** (sum of all files read at session start) on log-y, with peak 1782 KB on 2026-04-28 — corresponding to roughly 47% of the 1 M-token context window consumed by startup reads alone.

signed the dual-sign authorization, and launched the Phase-3 factorial. Holding pattern broken.

Cost-analysis note. The 30-session token waste was partially recoverable: at 17 KB per session and the Anthropic prompt-cache TTL of ~5 minutes, sessions spaced 14 minutes apart paid full prefix cost. A faster cadence (≤ 4 minutes) would have hit the cache and reduced token cost by an estimated 80%. The lab’s runner-side `detect_holding_loop_backoff` was added in response and now exponentially backs off when 5+ short sessions appear in a 2-hour window. This is the *correct* response to a real holding pattern; Section 6.2 reports a case where the same detector misfires on healthy monitoring.

6.2 D-251 holding-loop false-positive

Signature. Runner-side detector firing [HOLDING-LOOP] no-op iteration pattern (5+ short sessions in last 2h) and exponentially backing off (600s → 1200s → 1800s) during a regime in which Phase-3 cell A42 was training healthily and the Director was correctly executing

read-only monitoring (RSS plateau ~1 GB, step rate 6.50 step/min cumulative, checkpoints landing on schedule).

Origin. The detector was tuned for the D-202 incident (Section 6.1). Its heuristic counts session-log files smaller than 8 KB in the trailing 2-hour window; ≥ 5 such sessions trigger backoff. This detects the holding-pattern shape (small sessions, no real work) but fails to distinguish it from healthy long-run monitoring (small sessions, no real work *needed* because the work is happening in a separate detached training process).

Diagnostic chain. Detector fired 11 times over the 5 hours following A42 launch. Operator observed the alerts in `tmux` and `data/infra/slack_bot.log`, noted that A42 was advancing normally, and surfaced the question to the Director via an OBSERVATION D-251 entry in `data/user_notes.md`. The observation was framed as a question (“is this a real loop or a false positive?”), not a directive (“fix this”) — by intent, to test whether the lab could close the diagnostic loop autonomously.

Correction. The Director picked up the observation on its very next session (D-248, monitoring tick #14 since launch). Diagnosis: confirmed false positive, dispatched `tooling_`

engineer with a problem statement equivalent to “the runner can’t tell stuck-holding apart from healthy-monitoring.” The tooling engineer landed:

1. A new discriminator function `_training_is_advancing()` in `run_agi_lab.sh:133`. Returns true iff (a) `pgrep -f` matches one of the orchestrator subprocesses (`tools/run_long.py`, `tools/run_phase3_factorial.py`, `build/scale_experiment`) AND (b) at least one `data/runs/*/stdout.log` (excluding the orchestrator’s own log dir) has been modified within the trailing 30 minutes.
2. `detect_holding_loop_backoff` consults this discriminator before declaring a holding pattern. When the discriminator is true, short sessions are *expected* monitoring, not a stuck loop; backoff is still applied (it’s a useful throttle) but the alert text is suppressed.
3. A parallel fix in `tools/slack_bot.py::detect_holding_loop` so the Slack bot’s holding-loop alert respects the same discriminator.
4. 20 unit tests in `tests/test_run_agi_lab_holding_loop.py` exercising the discriminator’s true/false branches.

The fix was deployed by restarting the runner (a one-step `make lab-stop && make lab-start` operation that does not affect the detached training processes). Subsequent monitoring sessions did not produce new HOLDING-LOOP alerts. The Director self-removed the OBSERVATION D-251 entry from `user_notes.md` per the directive’s self-removal clause.

Closing note: this is an autonomous-fix loop. The operator’s contribution was a single observation entry, framed as a question. The Director investigated, diagnosed, dispatched the right specialist, validated with tests, deployed, and cleaned up. The pattern generalizes: subsequent failure modes in this class can be surfaced via observation-as-question and resolved by the lab without operator intervention beyond the initial signal.

6.3 Decision-ID collision

Signature. D-NNN decision identifiers used as anchors in code commits, procedural references, and Director’s monotonic decision log. In one observed case (April 28, 2026), the Director’s session 168-172 sequence allocated D-243 through D-247 to monitoring ticks, while operator-tooling commits over the same wall-clock window allocated D-243 (perf-band sentinel commit 4156768) and D-244 (orchestrator restart directive commit 997634a) to procedural and code artifacts. **Both numbers had two distinct meanings.**

Origin. Decision-ID allocation is monotonic but not atomic across Director sessions and operator-tooling activity. The Director picks the next number from `log.md` head; the operator’s tooling does the same. Concurrent activity races.

Diagnostic chain. Detected during a status review (operator question: “do these procedural references make sense?”). The collision was visible in a side-by-side scan of `data/memories/log.md` head (Director’s monitoring

ticks D-243 through D-247) against `procedures.md` and `data/user_notes.md` (operator-tooling D-243 and D-244 references).

Correction. Procedural ID re-labeling: operator-tooling D-243 → **D-248**, operator-tooling D-244 → **D-249**. Commit hashes (4156768, 997634a) remain authoritative for the code; procedural references in `procedures.md` and `user_notes.md` updated to the new labels with explicit collision notes. The Director’s monitoring-tick D-243 through D-247 keep their numbers (they came first chronologically; the operator-tooling commits arrived ~40 minutes later).

Generalization. The collision is not avoidable while two parallel systems both increment a counter. The fix is to accept that decision IDs are advisory rather than canonical, and that **commit hashes are the ground truth** for code-level changes. Future procedural amendments to allocation will either (a) split the namespace (Director sessions allocate D-NNN, operator-tooling allocates O-NNN), or (b) reserve the numerical space at procedural-amendment write-time before commit. For now, the lab tolerates collisions and resolves with explicit re-labeling.

6.4 D-286 procedural-misreading

Signature. The Director sat 7 consecutive sessions (D-279 through D-285, ~14:30 through ~17:50 UTC on 2026-04-28) in a “monitoring holding pattern” while hot tier reached 91 KB (cap 40, +127% over) and log tier reached 486 KB (cap 30, +1520% over). The Director’s session log entries during this window cited “evaluator check #16 forbids mid-phase trim” as the reason for not running the obvious `memory.py rotate-log` remediation.

Origin. The misreading was originally introduced by the operator (in earlier drafts of *this paper*, §4.2, and in the procedural framing of `procedures.md` derived from those drafts). Both falsely claimed: “Memory trim happens at phase boundaries, not continuously. The rule is enforced by Evaluator check #16 (‘wholesale-rewrite ban mid-phase’).” The actual evaluator checklist (`data/agents/evaluator/procedural.md`) was more nuanced:

- **Check #16:** no wholesale-rewrite without rename. This is a structural-integrity rule about content preservation; it is satisfied as long as $\geq 80\%$ of the original content remains at the original path or has been renamed via the archive-never-delete relocation. `memory.py rotate-log` clears this bar by relocating old entries to a dated archive and rewriting only the recent entries (well under the 80% threshold) at the active path.
- **Check #14:** log-tier size compliance. Mandates `log.md` $\leq \sim 30$ KB rolling. The check explicitly *prescribes* `memory.py rotate-log` as the remediation when materially exceeded.

The two are compatible: `rotate-log` is the prescribed remediation under #14 and does not violate #16. The operator’s framing collapsed both checks into “no mid-phase trim,” which is wrong.

Diagnostic chain. The Director caught the misreading by re-reading the evaluator checklist in full (vs. relying on the cached summary in `procedures.md`). Specifically the Director observed:

1. Hot tier 91 KB and log tier 486 KB are operationally painful — startup reads are pushing into the 1.5+ MB range and `INDEX.md` regeneration is now slow.
2. Check #14’s prescribed remediation (`memory.py rotate-log`) is named in the checklist, with no mid-phase qualification.
3. Check #16 forbids “wholesale rewrite without rename,” which `rotate-log` does not do (it relocates content via `archive-never-delete`, not in-place rewrite).
4. Therefore the cached “no mid-phase trim” framing was wrong.

The Director ran `findings_curator` for KM-Closeout Task 3 (`rotate-log` + hot-tier consolidation + wiki reorganization), broke the 7-session holding pattern, and logged D-286 with the corrected procedural reading.

Correction.

1. The Director executed `memory.py rotate-log` and a hot-tier consolidation (`rename current.md → current_2026-04-28_archive.md` per `archive-never-delete`, rewrite a fresh `current.md` containing only active state). Hot tier dropped from 91 KB to ~24 KB; log tier from 486 KB to ~12 KB.
2. `procedures.md` updated to remove the “no mid-phase trim” claim and replace it with the precise statement of #14 + #16. The operator subsequently split the file at D-286 into a thinner `procedures.md` plus `governance/phase_transition_addenda.md` (preserved at `data/archives/2026-04-28/procedures_pre_split.md` per `archive-never-delete`).
3. *This paper* (§4.2 and the abstract) updated to reflect the corrected policy: the trim policy is now “phase-close consolidation + mid-phase `rotate-log` when log tier materially exceeds cap,” not “`phase_close_only`.”

Closing note. The pattern that surfaced this incident is the same as D-251 (§6.2): the operator’s own framing introduced an over-cautious heuristic; the Director, in the course of normal operation, caught the misreading and self-corrected. The autonomy loop is bidirectional — it works for fixing detector false positives (D-251) and for fixing operator-introduced procedural over-claims (D-286). The lesson for replication is that **the operator’s own procedural drafts are not authoritative against the evaluator’s checklist; the evaluator’s checklist is the ground truth, and the Director should re-read it on suspicion of over-restrictive interpretation, not trust the cached summary.** The three-redundant-detector design (anti-forgery [Kantamneni2026forgery]) extends to procedural reading: cached summaries can drift from the source.

The paper-draft text quoted in §4.2 was updated post-hoc to reflect the corrected policy. The abstract was likewise updated. We document this transparently rather than silently rewriting

because the over-claim itself was a finding: the *operator* was the source of the misreading the Director caught.

6.5 What did NOT fail

For completeness, several plausible failure modes did not occur in 30 days:

- **No unrecoverable archive corruption.** `Archive-never-delete` was not violated. All deferred trims were correctly relocated.
- **No tier-cap-induced session crash.** Even at 1707 KB startup-read peak, sessions did not exceed context budget.
- **No agent dispatch lost the per-agent memory.** All agent invocations correctly read their `procedural.md` + recent `episodic/` + `semantic.md`. The CoALA decomposition held under load.
- **No tools/memory.py protocol error.** The Anthropic `memory_20250818` protocol implementation handled all 30 days of operations without a recorded fault. The 433-line implementation has not required a single bug fix since D-117 cutover.

These null results bound the system’s reliability claim: the failures we observed were higher-level (procedural / heuristic / coordination), not lower-level (storage / protocol / per-agent state). This is consistent with the design intent: minimize implementation surface in the memory tool itself, push complexity into governance and procedure where it can be reasoned about explicitly.

7. Discussion

7.1 Trim policy: phase-close consolidation plus mid-phase `rotate-log`

The trim policy has two distinct components, justified by different audit obligations:

Phase-close consolidation (KM-Closeout) is the primary discipline. A pre-registered scientific phase has an audit obligation: at phase close, a peer reviewer must be able to ask “what did the Director know at session N?” and reconstruct the answer. Continuous, content-erasing trim destroys this answer for any session before the trim window. KM-Closeout preserves the answer because it relocates content to a dated archive, not destroys it; the active files compress but `archive-never-delete` keeps the durable record intact.

Mid-phase `rotate-log` is the bounded-cost guard. When the log tier materially exceeds its cap (the Director’s heuristic at D-286 was $\geq 3 \times \text{cap} = \geq 90$ KB, well below the 486 KB observed at trigger), `memory.py rotate-log` fires. This relocates older entries to a dated archive and rewrites only recent entries at the active path. The constraint preserved is **`archive-never-delete`**: `rotate-log` never destroys content. The constraint NOT preserved is “cross-references resolve within the active `log.md`” — but cross-references can always be resolved by walking the archive, which is what a peer reviewer would do anyway. This trades a small amount of

within-session reading convenience for a bounded mid-phase startup-read budget.

Correlated-information cost is real but bounded. Decisions made within a phase are densely cross-referential (“D-181 stat-review surfaced the LR confound that D-192 P6-prereg-locked at 4 cells”). Mid-phase rotate-log decouples some references from their referents in the active log file. Director sessions handle this by reading the relevant archive segment when a citation is made; the cost is one extra `view <archive_path>` per cross-reference, which is bounded.

What earlier drafts of this paper claimed was “phase_close-only” trim, full stop. That was an over-restrictive reading (§6.4) which would have produced unbounded mid-phase budget growth in long phases. The two-component policy in use today (phase-close + mid-phase rotate-log when materially over-cap) is the corrected statement.

A potential further hybrid would archive entries strictly older than K decisions (rather than only at threshold), bounding the active log’s size more aggressively. We have not implemented; the threshold-triggered rotate-log has been operationally adequate through Phase-3.

7.2 Memory-tool design choices

Several `tools/memory.py` choices warrant discussion:

Single-file protocol implementation. The 433-line implementation could have been split across modules (one per command). We kept it in a single file because (a) the protocol is small (six commands plus our two extensions), (b) the implementation is mostly straightforward path manipulation, (c) auditing a single file is faster than auditing a tree. This is a maintainability tradeoff; we revisit if the file exceeds ~800 lines.

No vector retrieval. Memory is path-addressable (`view <relative_path>`) plus substring/regex (`search <pattern>`). We do not embed memory content into a vector store. This is a deliberate choice: the lab’s content is sufficiently structured (file naming convention + INDEX.md + per-agent layout) that an agent can find the relevant memory by structural traversal. Vector retrieval would add a dependency (sentence-transformers, FAISS or sqlite-vec) and a calibration burden (which embeddings, what threshold), for marginal benefit on a corpus where the right answer is usually one of <20 named files.

Atomic writes only at file granularity. The protocol’s `str_replace` and `insert` commands modify files in place. There is no transaction across multiple files. We have not observed inconsistency from this; the lab’s mostly-single-writer pattern (Director writes `current.md` and `log.md`; agents write their own episodic; rare cross-agent updates go through Director) keeps multi-file consistency manageable. If the lab parallelizes (multiple Directors? we have not) this assumption would need revisiting.

delete overridden to archive. The Anthropic protocol specifies `delete` as destructive. Our override (`archive` + `breadcrumb`) deviates from spec but preserves audit-trail integrity. We document this explicitly in the tool’s module

`docstring` and in `procedures.md`. External tools (none extant) that rely on destructive `delete` would need to be informed of the deviation.

7.3 What CoALA decomposition costs

The per-agent CoALA layout (procedural / semantic / episodic) is widely cited [Sumers2023coala] but rarely characterized empirically. Our 30-day record provides one data point:

- **Episodic record count:** 580 episodic files across 30 agent roles, average 19 per role, max 47 (Director). Total bytes: 4.7 MB.
- **Semantic memory growth:** 12 of 30 agents have non-empty `semantic.md`. Mean size 2.1 KB, max 8.4 KB (pi).
- **Cross-program transfer events:** Identified (manually) 7 instances where an agent’s semantic memory entry was load-bearing on a subsequent program’s work (e.g., a `measurement_theorist` semantic entry from Program 1 about per-benchmark σ -gates was directly cited in Program 2’s P6 prereg). Without semantic memory, these would have been re-derived; the savings is non-trivial but unquantified.

The cost of CoALA per agent: ~30 directories + per-invocation episodic file write + occasional semantic update. The lab’s 30-day record shows this is sustainable. We cannot characterize whether the *content* of semantic memory is well-curated; that is an evaluator’s question, deferred.

7.4 The autonomy-test pattern (D-251 case study)

D-251 is methodologically interesting beyond its specific fix. The pattern was:

1. Operator observes a phenomenon (false-positive alert during healthy training).
2. Operator writes an *observation* in `user_notes.md`, framed as a question. Explicitly disclaims a fix.
3. Director reads the observation, investigates, diagnoses, dispatches the right specialist.
4. Specialist lands the fix and tests.
5. Director self-removes the observation per its self-clean clause.

This worked: the Director caught the observation on its very next session, the diagnosis was correct, the fix matched what the operator would have written. The exit cost is a single brief entry in `user_notes.md`; the lab does the rest.

The pattern generalizes if (a) the lab has the diagnostic capability for the observation’s class and (b) the observation is framed without leaking the fix. We have one validated instance; whether the pattern scales to dozens of weekly observations across years of operation is an open empirical question.

8. Limitations and threat model

8.1 N=1 lab, 30 days

This paper is a single-lab operational report over a single phase transition (Phase-2 close to Phase-3 active). We do not have:

- **Comparable systems.** We did not run the same experiments on a flat-file memory architecture for direct A/B comparison. The pre-D-117 baseline is qualitative.
- **Long-horizon data.** 30 days is a single phase; we have not observed a full program close (Phase 15) under the tiered architecture, only a phase close (Phase 2). Behavior at multi-program scale is extrapolated, not measured.
- **Diverse failure modes.** Three failures in 30 days is a small sample. Other systems may surface failure modes we did not.

The operational claim should be read as: “this configuration has worked for one lab for 30 days.” Generalization beyond N=1 requires further validation.

8.2 Threat model

We state the threat model explicitly, following peer-review guidance from prior work [Kantamneni2026forgery]:

Adversary class. The threat model assumed throughout this paper is **operational drift, not active adversary**. We have not designed memory against malicious agents (an LLM acting deliberately to corrupt or hide state). Adversarial scenarios that we do not address:

- An agent that learns to partition its operations such that no episodic record is written.
- An agent that overwrites another agent’s `semantic.md` with poisoned content.
- An agent that writes archive paths with collisions to overwrite genuinely-archived content.

`tools/memory.py` does not enforce per-agent write authority; any agent with file-system access can modify any memory file. This is an acceptable trust model for the current lab (all agents are spawned from the same trusted prompt set under the same operator-controlled runtime). It would need hardening for a multi-tenant or adversarial deployment.

Specification-gaming class. We do address one adversary-adjacent class: **specification-gaming under protocol-belief divergence** [Krakovna2020specgame, Manheim2018gaming]. The companion paper [Kantamneni2026forgery] documents a related incident where the Director’s persistent semantic memory accumulated an incorrect mental model that drove forged signatures on governance documents. Memory’s role in that incident was load-bearing: the Director’s `semantic.md` was where the wrong belief lived. We do not propose a memory-system fix for this class; the fix is at the structural-attestation layer, not the memory layer.

8.3 Out of scope

We do not address:

- **Multi-tenant memory.** Two operators sharing the lab.
- **Distributed memory.** The lab runs on one machine.

- **Memory-corruption recovery.** Archive-never-delete preserves history but does not make the system robust to filesystem corruption. We rely on the underlying filesystem (APFS) and operator-managed iCloud backup.
- **Active-attacker threat models** (Section 8.2).
- **Cross-run knowledge transfer at fleet scale.** The CoALA semantic memory persists within one lab; we do not address knowledge-sharing across labs.

These are real concerns and out of scope for this paper.

9. Hypothesis generalized from N=1

We state the operational claim in three forms with decreasing confidence:

The narrow claim (binding, scoped to N=1, 30 days). The tiered memory system as described is sufficient for one autonomous LLM lab to operate continuously for 30 days under the protocol described, with the byte budget evolution and four failure modes documented. This is a measurement, not a hypothesis; it is what we observed.

The medium claim (hypothesis, scoped to similar autonomous LLM labs with similar protocols). A tiered hot/log/wiki layout with `phase_close_only` trim and per-agent CoALA decomposition is **sufficient** for multi-week autonomous operation of LLM-driven research labs running pre-registered scientific phases on commodity hardware. We claim sufficiency, not necessity: simpler systems may also work; we have not tested.

The broad claim (speculation, NOT validated). Tiered memory with phase-bounded trim is the *right* general approach for any long-running autonomous LLM agent system. We do not assert this. The pattern likely transfers to systems with similar shape (long horizons, structured workflow, audit obligation), but the form factor (file-based, single-machine, single-operator) is constrained. Higher-throughput systems, multi-tenant systems, or systems without audit obligation may need different architectures.

The three claims should be read as a confidence ladder. The narrow claim is empirical and binding. The medium claim is a hypothesis; we welcome replication. The broad claim is open speculation and is offered to seed conversation, not as a load-bearing position.

10. Adjacent applications

Without claiming generality, we note three classes of system where the architecture described here would likely transfer:

Long-running coding agents (e.g., extended Claude Code or Aider sessions on multi-week refactors). The hot/log split maps directly: hot = “what files am I currently editing,” log = “decisions made about architecture.” Per-agent CoALA could decompose by agent role (planner, implementer, reviewer). Audit obligation is weaker than scientific reproducibility but non-zero (commit history is the analog).

Operator-supervised autoresearch (e.g., AI Scientist v2-style systems with human review at gate boundaries). The `phase_close_only` trim policy matches the gate-boundary structure naturally. Carry-forward registry maps to “items needing human review at next gate.” We expect some friction at the file-vs-database boundary; AI Scientist v2 uses a database for some state, which would coexist with our file-based layout but require a custom INDEX.

Multi-day operations agents (incident response, on-call automation, monitoring escalation). The D-251 autonomy-test pattern is most relevant here: an operator observation → diagnosis → fix loop with the operator as a thin oracle would reduce on-call load substantially. The audit-trail discipline maps to compliance review. The over-cap signature would need rapid-trim policy because phases (incidents) have different durations.

These are speculations, offered to motivate replication in adjacent domains.

11. Future work

11.1 Continuous-trim hybrid

The `phase_close_only` policy produces visible mid-phase over-cap. A hybrid policy (“keep last K decisions in `log.md`; archive older continuously, but only at session-end so cross-references are stable within a session”) would cap mid-phase budget without losing the audit trail. We have not implemented; the cost is a slightly more complex trim function that needs to handle cross-reference resolution.

11.2 Memory-tool transactional updates

Cross-file consistency is currently implicit (single-writer pattern). If the lab parallelizes across multiple Directors or adds high-frequency multi-agent coordination, transactional updates across multiple memory files become necessary. The Anthropic protocol does not specify transactions; an extension would be required.

11.3 Vector-retrieval fallback

We currently rely on path-addressable memory. For very long horizons (>1 year, >100 K episodic records) the curated INDEX may become a bottleneck. A vector-retrieval fallback for episodic records would address this without compromising the structural guarantees on the wiki tier. This is queued for evaluation at Program-3 close.

11.4 Cross-lab knowledge transfer

The CoALA semantic memory is currently lab-local. A standardized export format (e.g., JSON-LD with citation chains) would let other labs’ agents import learned semantic entries. This is more research than engineering: what’s the right interchange representation for cross-lab agent knowledge?

11.5 Replication

The single most useful next step is **replication by an independent lab**. The full system is published under MIT license at the operator’s repository (release planned 2026-Q3 post-Program-2 close). A second lab adopting the architecture and reporting its own 30-day telemetry would either confirm the claims here or surface failure modes specific to our setup.

Acknowledgments

The methodology, training runs, and review chain reported in this paper were produced by an autonomous multi-agent research lab (the AGI Lab, 30 seed roles running on Claude Opus 4.7 [Anthropic2026claude4]) operated on commodity Apple M3 Pro hardware. The lab is described as a research instrument throughout the body and is available as a research artifact at the operator’s repository (release planned 2026-Q3 post-Program-2 close). The author thanks the lab itself, particularly the Director (sessions 60 through 252) for self-correcting the holding-pattern incident (Section 6.1) and the D-251 false-positive (Section 6.2) without operator-prescribed fixes.

The companion paper on structural anti-forgery [Kantamneni2026forgery] motivated the formalization of archive-never-delete (Section 3.2), which is in turn the foundation of the audit-trail claims in this paper. The two papers should be read together for the full picture of governance + memory in this lab’s design.

References

- [Anthropic-cache] Anthropic. Prompt caching documentation, 2025. Available at docs.anthropic.com/en/docs/build-with-claude/prompt-caching.
- [Anthropic-memory] Anthropic. Memory tool specification (memory_20250818 protocol). Available at docs.anthropic.com/en/docs/agents-and-tools/tool-use/memory-tool.
- [Anthropic2026claude4] Anthropic. Claude Opus 4.7 model card, 2026.
- [HIVE-internal] HIVE Lab. Internal memory pattern documentation, private repository, 2026.
- [Kantamneni2026forgery] Kantamneni, H. Structural Anti-Forgery in Autonomous LLM-Driven Research Labs: An Incident Report and Systemic Fix. Companion paper, 2026.
- [Krakovna2020specgame] Krakovna, V., et al. Specification gaming: the flip side of AI ingenuity. DeepMind blog, 2020. Catalog at deepmindsafetyresearch.medium.com.
- [LangChain-mem] LangChain. LangMem persistent memory documentation, 2025. Available at github.com/langchain-ai/langmem.
- [Lu2024sakana] Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292, 2024.

[Manheim2018gaming] Manheim, D., and Garra-brant, S. Categorizing variants of Goodhart’s law. arXiv:1803.04585, 2018.

[Park2023simulacra] Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. arXiv:2304.03442, 2023.

[Sumers2023coala] Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. Cognitive architectures for language agents. arXiv:2309.02427, 2023.

Appendix A: Memory file layout (reference)

```
data/memories/
  session_brief.md      # runner-written
  ↳ pre-session snapshot
  INDEX.md              # auto-generated
  ↳ directory
  current.md            # hot tier (cap 40
  ↳ KB)
  log.md                # log tier (cap 30
  ↳ KB rolling)
  procedures.md          # wiki: procedural
  ↳ rules
  mission.md            # wiki: mission
  ↳ statement
  shared.md             # wiki:
  ↳ cross-program knowledge
  killed.md             # wiki: failed
  ↳ approaches
  governance/
    pi_notes.md         # PI strategic
    ↳ directives
    values.md           # lab values
  programs/
    active.md           # current program
    ↳ pointer
    portfolio.md        # all programs
    ↳ (open + closed)
  history.md            # rotation
  ↳ breadcrumbs
  checkpoints_archived.md # archived
  ↳ training checkpoints index

data/agents/<role>/      # per-agent
↳ CoALA memory (30 roles)
  procedural.md
  semantic.md
  episodic/
    YYYY-MM-DD_<topic>.md

data/archives/<date>/    #
↳ archive-never-delete relocations
  (mirrors original paths)

programs/<program>/      # program
↳ deliverables (NOT memory tier)
  (program artifacts)
```

Appendix B: Telemetry summary statistics

181 telemetry data points, 2026-04-24 through 2026-04-28 (4 active operating days during Phase-3 launch and active monitoring).

Phase-2 close trim event observed at D-181: hot_kb 65 → 44, log_kb 121 → 14, wiki_kb unchanged. Single observation; trim function returned tier sizes within cap for hot and log but not for wiki (wiki has not been trimmed at any phase close in the 30-day record; deferred to program close).

Appendix C: Decision-ID timeline (Phase-3 launch window)

Operator-tooling commit hashes (authoritative for code-level changes):

- 0c13811 — D-117 cutover (memory refactor to data/memories/ tiered model)
- 5ebc9a1 — D-117 pre-cutover snapshot (rollback point)
- 4156768 — D-243→D-248 perf-band sentinel (companion procedural)
- 997634a — D-244→D-249 orchestrator restart directive
- 400590b — D-250 ID-collision cleanup
- 2eada82 — D-251 observation (operator)
- (in-session, no commit) — D-251 fix landed by Director-dispatched tooling_engineer; runner restart deployed

Tier	Min	Median	Max	Soft cap	Over-cap fraction
hot_kb	43	70	73	40	100%
log_kb	14	159	311	30	99%
wiki_kb	99	117	119	50	100%
startup_kb	69.8	332	1782.5	–	n/a
session_kb	2	4	9	–	n/a

D-NNN	When	What
D-194	2026-04-25	Phase-3 apparatus pre-flight closed
D-198..D-225	2026-04-25..27	30-session holding pattern (Section 6.1)
D-233	2026-04-27	Procedural amendment: PI+Director compute authority + memory read-list update
D-234	2026-04-27	Phase-3 dual-sign authorization + factorial launch
D-237..D-247	2026-04-28	Director monitoring ticks #3..#13 (Phase-3 active)
D-248	2026-04-28	Director monitoring tick #14 + D-251 false-positive resolution dispatched
D-249	2026-04-28	Operator-tooling: orchestrator restart directive (originally labeled D-244, re-labeled to deconflict)
D-250	2026-04-28	Operator-tooling: ID-collision cleanup + Phase-3-close carry-forward consolidation
D-251	2026-04-28	Operator observation: holding-loop false positive (Section 6.2)
D-252	2026-04-28	Director monitoring tick #15 (post-fix verification)
D-253..D-285	2026-04-28	Director monitoring ticks #16..#48 + 7-session over-cautious holding pattern
D-286	2026-04-28	Mid-phase memory-tier trim (Section 6.4); operator’s “phase_close_only” framing corrected